

State Space Models

Structured and Selective State Space Models for Sequence Modeling

Yasser Hifny

Faculty of Computers and Artificial Intelligence
Capital University

Table of Contents

- 1 Discrete-Time State Space Model
 - Training SSMs
- 2 HiPPO Initialization for S4 Models
- 3 Selective State Space (S6) Models
 - Bidirectional Mamba models
- 4 Mamba Evolution: From Mamba-1 to Mamba-3
 - Interpretation of Δ_k
 - Simplified Mamba-1 Block
 - Numerical Example of Mamba-1
 - Mamba-2
 - Mamba-3
 - Comparison of Mamba-1, Mamba-2, and Mamba-3
- 5 Improvements based on Mamba

Overview

- Continuous-time and discrete-time state space models
- Bilinear discretization of a linear time-invariant system
- Training state space models as a global convolution
- HiPPO initialization for long-range dependencies
- Selective state space (S6) models and the Mamba architecture
- The parallel scan algorithm for efficient training
- The Mamba family: Mamba-1, Mamba-2, and Mamba-3
- Numerical walk-throughs for each Mamba variant
- Mixture-of-Experts and the Jamba block

Continuous-Time State Space Models

State space models (SSMs) use a linear recurrence mechanism, in contrast with the non-linear recurrent structures used by recurrent neural networks (RNNs).

A structured state space sequence model (S4) combines continuous-time state space models with structured parameterizations and HiPPO (High-order Polynomial Projection Operators) theory.

The core of S4 is a linear time-invariant (LTI) system.

Continuous-Time SSM

State Equation

The hidden state evolves according to

$$\frac{dh(t)}{dt} = Ah(t) + Bx(t)$$

- $h(t) \in \mathbb{R}^N$: hidden state at time t
- $x(t) \in \mathbb{R}$: input at time t
- $A \in \mathbb{R}^{N \times N}$: state transition matrix
- $B \in \mathbb{R}^{N \times 1}$: input matrix

Output Equation

$$y(t) = Ch(t) + Dx(t)$$

- $y(t) \in \mathbb{R}$: output at time t
- $C \in \mathbb{R}^{1 \times N}$: output matrix
- $D \in \mathbb{R}$: skip connection (optional)

Bilinear Discretization

Approximating the Derivative

To use a discrete input sequence, the bilinear transformation discretizes the continuous-time state space model.

$$\frac{dh(t)}{dt} \approx \frac{h_k - h_{k-1}}{\Delta},$$

where Δ is the discretization step size (sampling interval).

Substituting into the State Equation

Substituting the approximation into the state equation:

$$\frac{h_k - h_{k-1}}{\Delta} = A \left(\frac{h_k + h_{k-1}}{2} \right) + B \left(\frac{x_k + x_{k-1}}{2} \right)$$

Rearranging for h_k

Rearranging terms to solve for h_k :

$$h_k - h_{k-1} = \frac{\Delta}{2} A(h_k + h_{k-1}) + \frac{\Delta}{2} B(x_k + x_{k-1})$$

Bilinear Discretization

Collecting h_k Terms

Collecting h_k terms on the left-hand side:

$$h_k - \frac{\Delta}{2} A h_k = h_{k-1} + \frac{\Delta}{2} A h_{k-1} + \frac{\Delta}{2} B(x_k + x_{k-1})$$

Factoring Out h_k

Factoring out h_k :

$$\left(I - \frac{\Delta}{2} A\right) h_k = \left(I + \frac{\Delta}{2} A\right) h_{k-1} + \frac{\Delta}{2} B(x_k + x_{k-1})$$

Solving for h_k

Solving for h_k :

$$h_k = \left(I - \frac{\Delta}{2} A\right)^{-1} \left(I + \frac{\Delta}{2} A\right) h_{k-1} + \left(I - \frac{\Delta}{2} A\right)^{-1} \frac{\Delta}{2} B(x_k + x_{k-1})$$

Bilinear Discretization: Approximating $x_k \approx x_{k-1}$

We approximate $x_k \approx x_{k-1}$:

$$h_k = \left(I - \frac{\Delta}{2}A\right)^{-1} \left(I + \frac{\Delta}{2}A\right) h_{k-1} + \left(I - \frac{\Delta}{2}A\right)^{-1} \Delta B x_k$$

Discretized State Update ($\bar{A}$, $\bar{B}$ Form)

Hence, the discretized state equation is

$$h_k = \bar{A}h_{k-1} + \bar{B}x_k,$$

where

$$\bar{A} = \left(I - \frac{\Delta}{2}A\right)^{-1} \left(I + \frac{\Delta}{2}A\right), \quad \bar{B} = \left(I - \frac{\Delta}{2}A\right)^{-1} \Delta B.$$

Discretized Output Equation

$$y_k = \bar{C}h_k + Dx_k,$$

where $\bar{C} = C$.

Zero-order hold (ZOH) alternative

Instead of the bilinear transform, ZOH discretization can be used:

$$\bar{A} = e^{A\Delta}, \quad \bar{B} = (\Delta A)^{-1}(\bar{A} - I)\Delta B, \quad \bar{C} = C.$$

ZOH assumes the sampled signal stays constant between sample points.

Training SSMs as a Global Convolution

Instead of processing sequences step-by-step (like RNNs), SSMs exploit their linear time-invariant (LTI) property to convert the recurrent state space equations into a global convolution kernel.

- This kernel allows parallel computation of outputs across the entire sequence.
- It drastically speeds up training compared to sequential recurrence.

Unrolling the Discretized System

The convolution kernel K is derived from the discretized system:

- At $k = 0$: $h_0 = \bar{B} \cdot x_0$, $y_0 = C\bar{B}x_0$
- At $k = 1$: $h_1 = \bar{A}\bar{B}x_0 + \bar{B}x_1$, $y_1 = C\bar{A}\bar{B}x_0 + C\bar{B}x_1$
- At $k = m$: $h_m = \bar{A}^m\bar{B}x_0 + \cdots + \bar{B}x_m$, $y_m = C\bar{A}^m\bar{B}x_0 + \cdots + C\bar{B}x_m$

SSM as a Convolution: Output Sequence

Thus, the sequence of outputs $\{y_0, y_1, y_2, \dots\}$ is given by (excluding Dx_k , handled separately):

$$y = x * K,$$

where K is the convolution kernel and $*$ denotes convolution.

SSM Convolution Kernel K

$$K = \left[C\overline{B}, C\overline{A}\overline{B}, C\overline{A}^2\overline{B}, \dots, C\overline{A}^{L-1}\overline{B} \right],$$

where L is the sequence length.

Efficient Convolution via FFT

The convolution $x * K$ is computed efficiently using the Fast Fourier Transform (FFT):

$$y = \mathcal{F}^{-1}(\mathcal{F}(K) \odot \mathcal{F}(x)) + Dx,$$

reducing complexity from $O(L^2)$ (naive convolution) to $O(L \log L)$.

Convolutional Training, Recurrent Inference

- During training, a convolutional formulation enables parallel computation across the full sequence, accelerating parameter updates and avoiding vanishing/exploding gradients.
- During inference, the model switches to a recurrent formulation, generating outputs step-by-step with minimal overhead.
- Precomputing K for long sequences requires significant memory, which can be mitigated by chunking.

Key idea

Convolutional training, recurrent inference: the same underlying LTI system supports both computational modes.

Why HiPPO Initialization Is Needed

A standard state space model demonstrates weak empirical performance and struggles to capture long-range dependencies effectively.

- Linear first-order ODEs inherently solve to exponential functions, leading to gradients that scale exponentially with sequence length.
- Repeatedly applying a recurrent matrix produces the familiar vanishing or exploding gradient problem also seen in RNNs.

The HiPPO-LegS Matrix

The HiPPO framework initializes A to optimally project historical input $x(t)$ onto polynomial bases (e.g., Legendre polynomials). The HiPPO-LegS matrix for the scaled Legendre basis is

$$A_{nk} = - \begin{cases} (2n+1)^{1/2}(2k+1)^{1/2} & \text{if } n > k, \\ n+1 & \text{if } n = k, \\ 0 & \text{otherwise.} \end{cases}$$

This allows the state $h(t)$ to compress the history of $x(t)$ into coefficients of an orthogonal polynomial basis.

Diagonal Plus Low-Rank (DPLR) Structure

To reduce computational complexity, A is constrained to a diagonal plus low-rank (DPLR) structure:

$$A = \Lambda - PP^*,$$

where $\Lambda \in \mathbb{C}^{N \times N}$ is diagonal and $P \in \mathbb{C}^{N \times r}$ is low-rank.

Benefit

DPLR enables $O(N)$ parameters instead of $O(N^2)$, with efficient computation via conjugate symmetry and the FFT.

Empirical Impact of HiPPO Initialization

Earlier research demonstrated that replacing a randomly initialized SSM matrix A with the HiPPO matrix significantly enhanced performance on the sequential MNIST classification benchmark:

Accuracy: 60% $\rightarrow$ 98%.

From Fixed to Selective SSMs

In traditional SSMs (e.g., S4), the convolution form arises because the system is linear time-invariant (LTI): A , B , and C are fixed and do not depend on the input or time.

Selective state space (S6) models, or Mamba models, introduce a selection mechanism to make the SSM more flexible and efficient.

- B and C become input-dependent, breaking the LTI assumption.
- The convolution form must therefore be adapted to account for the time-varying nature of B and C .

The Selection Mechanism: Learned Projections

The selection mechanism modifies B and C based on the input x_k through learned projections:

$$B_k = B \cdot \text{Linear}_B(x_k)$$

$$C_k = C \cdot \text{Linear}_C(x_k)$$

where Linear_B and Linear_C are learned linear transformations.

Selective SSM (Mamba)

State Update

Hence, the Mamba model is given by

$$h_k = \bar{A}h_{k-1} + \bar{B}_k x_k$$

Output Equation

$$y_k = C_k h_k + D x_k$$

Because B_k and C_k vary with time, the convolution kernel K is no longer fixed.

Selective SSM

Time-Varying Convolution View

Instead, the output y can be expressed as a time-varying convolution:

$$y_k = \sum_{i=1}^k C_k \bar{A}^{k-i} \bar{B}_i x_i$$

Here, the kernel is implicitly defined by the time-dependent terms C_k and $\bar{B}_i$, depending on both the current step k and all past steps i .

Sequential Recurrence Recap

Mamba computes the hidden states $\{h_k\}$ efficiently using a *parallel scan algorithm* (similar to prefix sums), enabling parallel training despite the sequential recurrence:

$$h_k = \bar{A} \cdot h_{k-1} + \bar{B}_k \cdot x_k.$$

This avoids $O(L^2)$ complexity and leverages modern hardware (GPUs/TPUs).

Unrolled Hidden State (Naive Expansion)

For a sequence of length L , the hidden state at position k can be unrolled explicitly:

$$h_k = \bar{A}^k h_0 + \sum_{j=1}^k \bar{A}^{k-j} \bar{B}_j x_j.$$

A naive parallel implementation would attempt to compute each h_k independently from this expression.

Complexity of a Single Unrolled Term

For each hidden state h_k , the summation term

$$\sum_{j=1}^k \overline{A}^{k-j} \overline{B}_j x_j$$

contains k contributions. Precomputing the powers $\overline{A}^1, \dots, \overline{A}^L$ independently for all pairs (k, j) can also grow to $O(L^2)$ operations.

Total Complexity of the Unrolled Formulation

Across all L hidden states, the total number of contributions is

$$\sum_{k=1}^L k = \frac{L(L+1)}{2} = O(L^2).$$

Thus, directly evaluating the unrolled formulation for every hidden state leads to quadratic complexity in the sequence length.

Sequential Recurrence: The $O(L)$ Alternative

In contrast, the sequential recurrence computes each state from only the immediately preceding state:

$$h_k = \bar{A}h_{k-1} + \bar{B}_k x_k,$$

requiring only one state update per sequence position, and therefore $O(L)$ sequential steps rather than $O(L^2)$.

The Parallel Scan

Associative Reformulation

The parallel scan algorithm leverages the associative property of linear recurrence to hierarchically compose transformations.

The recurrence can be viewed as applying a linear operator $T_k = (\bar{A}, \bar{B}_k x_k)$ to the state h_{k-1} . These operators compose associatively:

$$T_i \circ T_j = (\bar{A}, \bar{B}_i x_i) \circ (\bar{A}, \bar{B}_j x_j) = (\bar{A}^2, \bar{A}\bar{B}_i x_i + \bar{B}_j x_j),$$

with $(T_1 \circ T_2) \circ T_3 = T_1 \circ (T_2 \circ T_3)$.

Tree-Based Computation

The scan algorithm proceeds in $\log L$ layers:

- **Layer 1:** compose adjacent pairs $(T_1 \circ T_2), (T_3 \circ T_4), \dots$ in parallel.
- **Layer 2:** compose the Layer-1 results into chunks of four steps, and so on.

After $\log L$ layers, the full sequence is covered.

Work complexity

Each layer performs $O(L)$ compositions. With $\log L$ layers, the total work is $O(L \log L)$.

Parallel Scan Example

$L = 4$, Steps 1–3

- ① **Step 1:** compute local transformations T_1, T_2, T_3, T_4 .
- ② **Step 2:** compose adjacent pairs in parallel:

$$T_{1:2} = T_1 \circ T_2, \quad T_{3:4} = T_3 \circ T_4.$$

- ③ **Step 3:** compose results to cover the full sequence:

$$T_{1:4} = T_{1:2} \circ T_{3:4}.$$

Recovering Every h_k

To recover all h_k (not just h_4), intermediate results are stored:

$$h_1 = T_1 h_0, \quad h_2 = T_{1:2} h_0, \quad h_3 = T_{1:2} \circ T_3 h_0, \quad h_4 = T_{1:4} h_0.$$

Total operations

4 (Step 1) + 2 (Layer 1) + 1 (Layer 2) = 7 compositions.

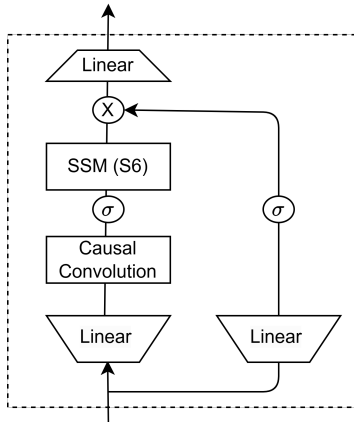
The parallel scan avoids $O(L^2)$ by reusing intermediate computations, exploiting associativity to hierarchically merge chunks, and achieving $O(L \log L)$ work instead of brute-force $O(L^2)$.

The Mamba Block

Selective SSM blocks can be integrated as independent transformation units within a neural network, similar to how RNN cells such as LSTMs or GRUs are utilized.

The complete structure of a Mamba block extends beyond the SSM module itself: it includes linear projections, convolutions, and non-linear activation functions that work alongside the SSM block within the broader Mamba architecture.

Figure: The Mamba Block Architecture



A Mamba block.

Transformers versus Mamba

- Transformers rely on attention to model long-range dependencies; Mamba relies on structured state space models (SSMs), making it more efficient for sequential tasks.
- Self-attention scales quadratically, whereas Mamba achieves linear scaling with sequence length.
- Inference: Transformers require $O(L)$ operations due to attention, while Mamba operates in constant time $O(1)$ per step.
- Training: Transformers have $O(L^2)$ complexity, whereas Mamba benefits from $O(L \log L)$ training complexity.

Transformer versus Mamba: Summary Table

Feature	Transformer	Mamba
Architecture	Attention-based	SSM-based
Complexity	High	Lower
Inference speed	$O(L)$	$O(1)$
Training speed	$O(L^2)$	$O(L \log L)$
Memory usage	$O(L^2)$	$O(L)$

Comparison between the Transformer and Mamba blocks.

Bidirectional Mamba (Bi-Mamba)

Bi-Mamba extends the strengths of Mamba (efficiency, selectivity) to bidirectional contexts, making it a powerful tool for tasks requiring full-sequence understanding.

- It sacrifices some autoregressive capability.
- It outperforms Transformers and RNNs in memory efficiency and long-context modeling for non-autoregressive applications.
- Bi-Mamba has two Mamba blocks: one handles the past context, the other handles the flipped input (future context) – analogous to bidirectional RNNs.

Bi-Mamba: Combining Forward and Flipped Outputs

$$y = \text{Mamba}(x) + \text{flip}(\text{Mamba}(\text{flip}(x)))$$

The Mamba Family

State space models provide an alternative to attention-based sequence models. The Mamba family (Mamba-1, Mamba-2, and Mamba-3) builds upon SSMs and progressively improves their

- selectivity,
- computational efficiency,
- discretization, and
- state representation.

Mamba-1

Exponential-Euler State Update

In the implementation-oriented formulation, Mamba-1 uses the exponential-Euler form:

$$\mathbf{h}_k = \exp(\Delta_k \mathbf{A}) \mathbf{h}_{k-1} + \Delta_k \mathbf{B}_k \mathbf{x}_k.$$

Fixed versus Input-Dependent Parameters

The most important innovation in Mamba-1 is that the SSM parameters depend on the input. Instead of fixed parameters

$$\mathbf{B}, \mathbf{C}, \Delta,$$

Input-Dependent Parameters

Mamba uses

$$\mathbf{B}_k, \mathbf{C}_k, \Delta_k = f(\mathbf{x}_k).$$

Mamba-1: State Update with Input-Dependent $\mathbf{A}_k$

Consequently, the state update becomes

$$\mathbf{h}_k = \mathbf{A}_k \mathbf{h}_{k-1} + \mathbf{B}_k \mathbf{x}_k.$$

This allows the model to dynamically determine

- what information should be written into the state,
- how long information should remain in the state,
- what information should be read from the state.

Interpretation of Δ_k

Scalar Recurrence

Consider the scalar recurrence

$$h_k = e^{\Delta_k A} h_{k-1} + \bar{B}_k x_k,$$

and assume $A < 0$.

Small Δ_k

When Δ_k is small,

$$e^{\Delta_k A} \approx 1,$$

so the previous state is largely preserved.

Large Δ_k

When Δ_k is large,

$$e^{\Delta_k A} \approx 0,$$

so the previous state is rapidly forgotten.

Key idea

Δ_k behaves as an input-dependent memory or time-scale parameter.

Simplified Mamba-1 Block

Input Sequence

For an input sequence

$$\mathbf{X} \in \mathbb{R}^{L \times d},$$

the input can first be projected:

$$\mathbf{U} = \mathbf{XW}_{\text{in}}.$$

Depthwise Convolution

A depthwise convolution is then applied:

$$\mathbf{V} = \text{Conv1D}(\mathbf{U}).$$

SiLU Activation

After applying the SiLU activation,

$$\mathbf{Z} = \text{SiLU}(\mathbf{V}),$$

the selective parameters are generated from $\mathbf{Z}_k$.

Simplified Mamba-1 Block

Generating Δ_k, B_k, C_k

$$\Delta_k = f_{\Delta}(\mathbf{Z}_k), \quad \mathbf{B}_k = f_B(\mathbf{Z}_k), \quad \mathbf{C}_k = f_C(\mathbf{Z}_k).$$

Gated SSM Output

The SSM output is combined with a gating branch. A simplified expression is

$$\mathbf{Y} = \text{SSM}(\mathbf{Z}) \odot \text{SiLU}(\mathbf{V}_{\text{gate}}).$$

Finally, an output projection is applied.

Full Pipeline

Thus, a simplified Mamba block is

$$\mathbf{X} \rightarrow \text{Projection} \rightarrow \text{Convolution} \rightarrow \text{Selective SSM} \rightarrow \text{Gate} \rightarrow \text{Projection}.$$

Mamba-1 Example

Setup

Consider a sequence of four scalar inputs:

$$x_1 = 1, \quad x_2 = 2, \quad x_3 = 3, \quad x_4 = 1.$$

Let the state have two dimensions, with

$$\mathbf{h}_k = \begin{bmatrix} h_{k,1} \\ h_{k,2} \end{bmatrix}, \quad \mathbf{h}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

State Matrix and Step Size

Assume

$$\mathbf{A} = \begin{bmatrix} -1 & 0 \\ 0 & -2 \end{bmatrix}, \quad \Delta_k = 0.5.$$

Discretized $\bar{\mathbf{A}}$

$$\bar{\mathbf{A}} = \exp(0.5\mathbf{A}) = \begin{bmatrix} e^{-0.5} & 0 \\ 0 & e^{-1} \end{bmatrix}.$$

Since

$$e^{-0.5} \approx 0.6065, \quad e^{-1} \approx 0.3679,$$

Mamba-1 Example

Discretized $\bar{\mathbf{A}}$ (Result)

we have

$$\bar{\mathbf{A}} = \begin{bmatrix} 0.6065 & 0 \\ 0 & 0.3679 \end{bmatrix}.$$

$\mathbf{B}_k$ and $\mathbf{C}_k$

Let

$$\mathbf{B}_k = \begin{bmatrix} 1 \\ 0.5 \end{bmatrix}, \quad \mathbf{C}_k = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

$\Delta \mathbf{B}$ Term

Using the exponential-Euler approximation,

$$\Delta \mathbf{B} = 0.5 \begin{bmatrix} 1 \\ 0.5 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 0.25 \end{bmatrix}.$$

Mamba-1 Example: General Recurrence

The recurrence is therefore

$$\mathbf{h}_k = \begin{bmatrix} 0.6065 & 0 \\ 0 & 0.3679 \end{bmatrix} \mathbf{h}_{k-1} + \begin{bmatrix} 0.5 \\ 0.25 \end{bmatrix} x_k.$$

Mamba-1 Example, Token 1

Hidden State

For $x_1 = 1$, since $\mathbf{h}_0 = \mathbf{0}$,

$$\mathbf{h}_1 = \bar{\mathbf{A}}\mathbf{h}_0 + \Delta\mathbf{B}x_1 = \begin{bmatrix} 0.5 \\ 0.25 \end{bmatrix}.$$

Output y_1

$$y_1 = \mathbf{C}^\top \mathbf{h}_1 = \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.25 \end{bmatrix} = 0.75.$$

Mamba-1 Example, Token 2

Decayed Previous State

For $x_2 = 2$,

$$\bar{\mathbf{A}}\mathbf{h}_1 = \begin{bmatrix} 0.6065 & 0 \\ 0 & 0.3679 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.25 \end{bmatrix} = \begin{bmatrix} 0.30325 \\ 0.09198 \end{bmatrix}.$$

Input Contribution

The input contribution is

$$\Delta \mathbf{B}_{x_2} = \begin{bmatrix} 0.5 \\ 0.25 \end{bmatrix} (2) = \begin{bmatrix} 1 \\ 0.5 \end{bmatrix}.$$

Hidden State $\mathbf{h}_2$

Therefore,

$$\mathbf{h}_2 = \begin{bmatrix} 1.30325 \\ 0.59198 \end{bmatrix}.$$

Mamba-1 Example, Token 2: Output y_2

The output is approximately

$$y_2 \approx 1.8952.$$

Mamba-1 Example, Token 3

Decayed Previous State

For $x_3 = 3$,

$$\bar{\mathbf{A}}\mathbf{h}_2 \approx \begin{bmatrix} 0.7904 \\ 0.2178 \end{bmatrix}.$$

Input Contribution

The input contribution is

$$\Delta \mathbf{B}_{x_3} = \begin{bmatrix} 1.5 \\ 0.75 \end{bmatrix}.$$

Hidden State $\mathbf{h}_3$

Therefore,

$$\mathbf{h}_3 \approx \begin{bmatrix} 2.2904 \\ 0.9678 \end{bmatrix}.$$

Mamba-1 Example, Token 3: Output y_3

$$y_3 \approx 3.2582.$$

Mamba-1 Example, Token 4

Decayed Previous State

For $x_4 = 1$,

$$\bar{\mathbf{A}}\mathbf{h}_3 \approx \begin{bmatrix} 1.3893 \\ 0.3559 \end{bmatrix}.$$

Hidden State $\mathbf{h}_4$

Adding the current input contribution gives

$$\mathbf{h}_4 \approx \begin{bmatrix} 1.8893 \\ 0.6059 \end{bmatrix}.$$

Output y_4

$$y_4 \approx 2.4952.$$

Observation

Across the four tokens, the hidden state accumulates a decayed memory of past inputs, and the output tracks this evolving state.

Mamba-2

State Space Duality

Mamba-2 introduces the concept of *State Space Duality (SSD)*. The central observation is that an SSM can be expressed in two mathematically related forms:

- 1 a recurrent state space formulation, and
- 2 a parallel structured matrix formulation.

Mamba-2 uses this relationship to construct an implementation that is much more efficient on modern GPU hardware.

Scalar-Times-Identity State Matrix

Mamba-2 uses a simplified state transition matrix:

$$\mathbf{A}_k = A_k \mathbf{I}.$$

Therefore,

$$\exp(\Delta_k \mathbf{A}_k) = \exp(\Delta_k A_k) \mathbf{I}.$$

Defining α_k

Define

$$\alpha_k = \exp(\Delta_k A_k).$$

Mamba-2

Simplified Recurrence

Then the recurrence becomes

$$\mathbf{h}_k = \alpha_k \mathbf{h}_{k-1} + \gamma_k \mathbf{B}_k \mathbf{x}_k,$$

where $\gamma_k = \Delta_k$.

Output Equation

$$\mathbf{y}_k = \mathbf{C}_k^\top \mathbf{h}_k.$$

Unrolling Mamba-2

Scalar-State Recurrence

Consider the scalar-state recurrence

$$h_k = \alpha_k h_{k-1} + \gamma_k B_k x_k.$$

$k = 1, 2$

For $k = 1$,

$$h_1 = \gamma_1 B_1 x_1.$$

For $k = 2$,

$$h_2 = \alpha_2 \gamma_1 B_1 x_1 + \gamma_2 B_2 x_2.$$

$k = 3$

$$h_3 = \alpha_3 \alpha_2 \gamma_1 B_1 x_1 + \alpha_3 \gamma_2 B_2 x_2 + \gamma_3 B_3 x_3.$$

Unrolling Mamba-2

$$k = 4$$

$$h_4 = \alpha_4 \alpha_3 \alpha_2 \gamma_1 B_1 x_1 + \alpha_4 \alpha_3 \gamma_2 B_2 x_2 \\ + \alpha_4 \gamma_3 B_3 x_3 + \gamma_4 B_4 x_4.$$

General Closed Form

In general,

$$h_k = \sum_{i=1}^k \left(\gamma_i \prod_{j=i+1}^k \alpha_j \right) B_i x_i.$$

This expression shows that the recurrent computation can be rewritten as a structured matrix operation.

Structured Matrix Form

Input Vector

For a sequence of four tokens, define

$$\mathbf{X} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}.$$

Lower-Triangular Matrix $\mathbf{L}$

The recurrence can be represented using a lower-triangular matrix

$$\mathbf{L} = \begin{bmatrix} \gamma_1 & 0 & 0 & 0 \\ \alpha_2 \gamma_1 & \gamma_2 & 0 & 0 \\ \alpha_3 \alpha_2 \gamma_1 & \alpha_3 \gamma_2 & \gamma_3 & 0 \\ \alpha_4 \alpha_3 \alpha_2 \gamma_1 & \alpha_4 \alpha_3 \gamma_2 & \alpha_4 \gamma_3 & \gamma_4 \end{bmatrix}.$$

State Space Duality: Recurrence $\Leftrightarrow$ Structured Matrix

Thus, the state computation can be viewed as a structured matrix operation. The key insight of State Space Duality is

Recurrent SSM $\Leftrightarrow$ Structured Matrix Operation.
--

This allows the computation to be organized around efficient matrix multiplication and chunked computation.

Chunking in Mamba-2

Sequence Length

Suppose a sequence of length L is divided into N chunks, each containing C tokens:

$$L = NC.$$

The computation can then be divided into intra-chunk computation and inter-chunk state propagation.

Intra- and Inter-Chunk Computation

Conceptually,

$$\text{Sequence} \rightarrow \text{Chunk}_1 \rightarrow \text{Chunk}_2 \rightarrow \dots \rightarrow \text{Chunk}_N.$$

This enables much more efficient use of GPU matrix-multiplication hardware.

Mamba-2 Example

State Matrix and Step Size

Consider

$$A = -1, \quad \Delta_k = 0.5.$$

Computing α_k

$$\alpha_k = e^{\Delta_k A} = e^{-0.5} \approx 0.6065.$$

Input Matrix $\mathbf{B}_k$

Let

$$\mathbf{B}_k = \begin{bmatrix} 1 \\ 0.5 \end{bmatrix}.$$

Mamba-2 Example

γ_k and $\gamma_k \mathbf{B}_k$

Since $\gamma_k = \Delta_k = 0.5$, we obtain

$$\gamma_k \mathbf{B}_k = \begin{bmatrix} 0.5 \\ 0.25 \end{bmatrix}.$$

Recurrence

Therefore,

$$\mathbf{h}_k = 0.6065 \mathbf{h}_{k-1} + \begin{bmatrix} 0.5 \\ 0.25 \end{bmatrix} x_k.$$

$\mathbf{h}_1$

For $x_1 = 1$,

$$\mathbf{h}_1 = \begin{bmatrix} 0.5 \\ 0.25 \end{bmatrix}.$$

Mamba-2 Example

h_2 (Setup)

For $x_2 = 2$,

$$\mathbf{h}_2 = 0.6065 \begin{bmatrix} 0.5 \\ 0.25 \end{bmatrix} + \begin{bmatrix} 1 \\ 0.5 \end{bmatrix}.$$

h_2 (Result)

Thus,

$$\mathbf{h}_2 = \begin{bmatrix} 1.30325 \\ 0.651625 \end{bmatrix}.$$

Mamba-3: Three Key Ideas

Mamba-3 further improves the SSM formulation through three major ideas:

Mamba-3 = Exponential-Trapezoidal Discretization + Complex-Valued States + MIMO.

These changes increase the expressiveness of the state dynamics while retaining the efficient recurrent and SSD formulations.

Exponential-Trapezoidal Discretization

Recap

Mamba-1 and Mamba-2 use an exponential-Euler-style update:

$$\mathbf{h}_k = \alpha_k \mathbf{h}_{k-1} + \gamma_k \mathbf{B}_k \mathbf{x}_k.$$

Adding the Previous Input

Mamba-3 introduces an additional contribution from the previous input:

$$\mathbf{h}_k = \alpha_k \mathbf{h}_{k-1} + \beta_k \mathbf{B}_{k-1} \mathbf{x}_{k-1} + \gamma_k \mathbf{B}_k \mathbf{x}_k.$$

Exponential-Trapezoidal Coefficients

α_k

$$\alpha_k = e^{\Delta_k A_k}.$$

β_k

$$\beta_k = (1 - \lambda_k) \Delta_k e^{\Delta_k A_k}.$$

γ_k

$$\gamma_k = \lambda_k \Delta_k.$$

The parameter λ_k is data-dependent and satisfies $0 \leq \lambda_k \leq 1$.

Exponential-Trapezoidal Discretization: Complete Recurrence

Therefore, the complete recurrence can be written as

$$\mathbf{h}_k = e^{\Delta_k A_k} \mathbf{h}_{k-1} + (1 - \lambda_k) \Delta_k e^{\Delta_k A_k} \mathbf{B}_{k-1} \mathbf{x}_{k-1} + \lambda_k \Delta_k \mathbf{B}_k \mathbf{x}_k.$$

This can be interpreted as an implicit width-two, data-dependent filtering operation.

Mamba-3 as a Width-Two Filter

Defining $\mathbf{v}_k$

Define

$$\mathbf{v}_k = \mathbf{B}_k \mathbf{x}_k.$$

The recurrence becomes

$$\mathbf{h}_k = \alpha_k \mathbf{h}_{k-1} + \beta_k \mathbf{v}_{k-1} + \gamma_k \mathbf{v}_k.$$

Dependence on $[\mathbf{v}_{k-1}, \mathbf{v}_k]$

Therefore, the current state depends on

$$[\mathbf{v}_{k-1}, \mathbf{v}_k].$$

This provides an implicit width-two convolution inside the SSM recurrence.

Mamba-3 Complex-Valued States

Complex A

Mamba-3 also considers complex-valued state dynamics. Let

$$A = \alpha + i\omega.$$

Then

$$e^{\Delta A} = e^{\Delta(\alpha + i\omega)}.$$

Euler's Identity

Using Euler's identity,

$$e^{i\theta} = \cos \theta + i \sin \theta,$$

Decay-Rotation Decomposition

we obtain

$$e^{\Delta A} = e^{\alpha \Delta} [\cos(\omega \Delta) + i \sin(\omega \Delta)].$$

This expression contains two important components.

Mamba-3 Complex-Valued States

Decay Term

The term

$$e^{\alpha\Delta}$$

controls the decay or memory of the state.

Rotation Term

The term

$$e^{i\omega\Delta}$$

introduces a rotation or oscillation.

Complex SSM = Decay + Rotation

Thus,

$$\text{Complex SSM} = \text{Decay} + \text{Rotation}.$$

Complex Example

State A and Step Size Δ

Suppose

$$A = -0.2 + 2i, \quad \Delta = 0.5.$$

ΔA

$$\Delta A = -0.1 + i.$$

Therefore,

$$e^{\Delta A} = e^{-0.1+i}.$$

Applying Euler's Identity

Using Euler's identity,

$$e^{-0.1+i} = e^{-0.1} (\cos 1 + i \sin 1).$$

Complex Example

Numerical Constants

Using $e^{-0.1} \approx 0.9048$, $\cos 1 \approx 0.5403$, $\sin 1 \approx 0.8415$,

$e^{\Delta A}$ (Result)
we obtain

$$e^{\Delta A} \approx 0.4885 + 0.7618i.$$

Previous State h_{k-1}

Suppose

$$h_{k-1} = 1 + i.$$

Then

$$e^{\Delta A} h_{k-1} = (0.4885 + 0.7618i)(1 + i) = 0.4885 + 0.4885i + 0.7618i + 0.7618i^2.$$

Complex Example

$$i^2 = -1$$

Since

$$i^2 = -1,$$

$e^{\Delta A} h_{k-1}$ (Result)

we obtain

$$e^{\Delta A} h_{k-1} \approx -0.2733 + 1.2503i.$$

Thus, the state is both attenuated and rotated.

Mamba-3 MIMO

Motivation

Mamba-3 extends the state space formulation toward a multi-input/ multi-output (MIMO) representation, in which multiple input channels can simultaneously influence the latent state and multiple output channels can be generated from the shared state.

This is more general than the conventional single-input/single-output (SISO) state space model.

Input, State, and Output Dimensions

Let the input, hidden state, and output at discrete position k be

$$\mathbf{x}_k \in \mathbb{R}^{d_{\text{in}}}, \quad \mathbf{h}_k \in \mathbb{R}^N, \quad \mathbf{y}_k \in \mathbb{R}^{d_{\text{out}}},$$

where d_{in} is the number of input channels, N is the latent state dimension, and d_{out} is the number of output channels.

Input-to-State Matrix $\mathbf{B}_k$

The input-to-state projection is represented by

$$\mathbf{B}_k \in \mathbb{R}^{N \times d_{\text{in}}}.$$

Mamba-3 MIMO

State-to-Output Matrix $\mathbf{C}_k$

The state-to-output projection is represented by

$$\mathbf{C}_k \in \mathbb{R}^{d_{\text{out}} \times N}.$$

State Transition Matrix $\mathbf{A}_k$

$$\mathbf{A}_k \in \mathbb{R}^{N \times N}.$$

State Update

The MIMO state update is therefore

$$\mathbf{h}_k = \mathbf{A}_k \mathbf{h}_{k-1} + \mathbf{B}_k \mathbf{x}_k.$$

Mamba-3 MIMO

Output Equation

The corresponding output equation is

$$\mathbf{y}_k = \mathbf{C}_k \mathbf{h}_k + \mathbf{D}_k \mathbf{x}_k,$$

Direct Feedthrough Matrix $\mathbf{D}_k$

where

$$\mathbf{D}_k \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$$

represents a direct input-to-output mapping.

Input-to-State Mapping

$\mathbf{B}_k \mathbf{x}_k$ Expanded

The matrix $\mathbf{B}_k$ determines how each input channel contributes to each state dimension:

$$\mathbf{B}_k \mathbf{x}_k = \begin{bmatrix} \sum_{j=1}^{d_{\text{in}}} B_{k,1j} x_{k,j} \\ \sum_{j=1}^{d_{\text{in}}} B_{k,2j} x_{k,j} \\ \vdots \\ \sum_{j=1}^{d_{\text{in}}} B_{k,Nj} x_{k,j} \end{bmatrix}.$$

Every state dimension can receive information from multiple input channels.

Two-Channel Example

Consider two input channels and two state dimensions:

$$\mathbf{x}_k = \begin{bmatrix} x_{k,1} \\ x_{k,2} \end{bmatrix}, \quad \mathbf{B}_k = \begin{bmatrix} 1 & 0.2 \\ 0.5 & 1 \end{bmatrix}.$$

Two-Channel Computation

The input contribution to the state is

$$\mathbf{B}_k \mathbf{x}_k = \begin{bmatrix} 1 & 0.2 \\ 0.5 & 1 \end{bmatrix} \begin{bmatrix} x_{k,1} \\ x_{k,2} \end{bmatrix} = \begin{bmatrix} x_{k,1} + 0.2x_{k,2} \\ 0.5x_{k,1} + x_{k,2} \end{bmatrix}.$$

Input-to-State Mapping

Numerical Input
For

$$\mathbf{x}_k = \begin{bmatrix} 2 \\ 3 \end{bmatrix},$$

Numerical Result
we obtain

$$\mathbf{B}_k \mathbf{x}_k = \begin{bmatrix} 2.6 \\ 4 \end{bmatrix}.$$

This illustrates the MIMO property: both input channels contribute to both state dimensions.

State-to-Output Mapping

$\mathbf{C}_k$

The matrix $\mathbf{C}_k$ maps the latent state to multiple output channels:

$$\mathbf{y}_k = \mathbf{C}_k \mathbf{h}_k, \quad \mathbf{C}_k \in \mathbb{R}^{d_{\text{out}} \times N}.$$

Numerical $\mathbf{C}_k$

For example, if

$$\mathbf{C}_k = \begin{bmatrix} 1 & 0.3 \\ 0.2 & 1 \end{bmatrix},$$

Numerical Result

then

$$\mathbf{y}_k = \begin{bmatrix} h_{k,1} + 0.3h_{k,2} \\ 0.2h_{k,1} + h_{k,2} \end{bmatrix}.$$

Each output channel can depend on multiple state dimensions.

Complete MIMO Mapping: Input-State-Output Pipeline

Combining the input-to-state and state-to-output transformations gives

$$\mathbf{x}_k \longrightarrow \mathbf{B}_k \mathbf{x}_k \longrightarrow \mathbf{h}_k \longrightarrow \mathbf{C}_k \mathbf{h}_k \longrightarrow \mathbf{y}_k.$$

Complete MIMO State-Space System

The complete state space system can be written as

$$\begin{aligned}\mathbf{h}_k &= \mathbf{A}_k \mathbf{h}_{k-1} + \mathbf{B}_k \mathbf{x}_k, \\ \mathbf{y}_k &= \mathbf{C}_k \mathbf{h}_k + \mathbf{D}_k \mathbf{x}_k.\end{aligned}$$

SISO System: Single Input-Output Path

In a SISO system, a single input affects a single output through the latent state:

$$x_k \rightarrow h_k \rightarrow y_k.$$

MIMO System: Multiple Inputs and Outputs

In a MIMO system, multiple inputs jointly influence the state, and the shared state jointly generates multiple outputs:

$$\begin{bmatrix} x_{k,1} \\ x_{k,2} \\ \vdots \\ x_{k,d_{\text{in}}} \end{bmatrix} \rightarrow \mathbf{h}_k \rightarrow \begin{bmatrix} y_{k,1} \\ y_{k,2} \\ \vdots \\ y_{k,d_{\text{out}}} \end{bmatrix} .$$

Consequently, $\mathbf{B}_k$ and $\mathbf{C}_k$ mix information across input and output channels, respectively.

Mamba-1 Recap

State Update and Contribution

$$\mathbf{h}_k = e^{\Delta_k \mathbf{A}} \mathbf{h}_{k-1} + \Delta_k \mathbf{B}_k \mathbf{x}_k$$

with input-dependent

$$\Delta_k, \mathbf{B}_k, \mathbf{C}_k.$$

Main Contribution

Main contribution: Selective State-Space Memory.

Mamba-2 Recap

State Matrix and α_k
Mamba-2 uses

$$\mathbf{A}_k = A_k \mathbf{I} \quad \text{and} \quad \alpha_k = e^{\Delta_k A_k}.$$

Recurrence and Main Contribution
Its recurrence becomes

$$\mathbf{h}_k = \alpha_k \mathbf{h}_{k-1} + \gamma_k \mathbf{B}_k \mathbf{x}_k.$$

Main contribution: State Space Duality (SSD).

Mamba-3 Recap

State Update
Mamba-3 uses

$$\mathbf{h}_k = \alpha_k \mathbf{h}_{k-1} + \beta_k \mathbf{B}_{k-1} \mathbf{x}_{k-1} + \gamma_k \mathbf{B}_k \mathbf{x}_k,$$

Coefficients
where

$$\alpha_k = e^{\Delta_k A_k}, \quad \beta_k = (1 - \lambda_k) \Delta_k e^{\Delta_k A_k}, \quad \gamma_k = \lambda_k \Delta_k.$$

Complex State Matrix

Mamba-3 additionally introduces complex-valued state dynamics,

$$A_k = \alpha_k + i\omega_k,$$

Mamba-3 Recap

Complex Matrix Exponential
so that

$$e^{\Delta_k A_k} = e^{\alpha_k \Delta_k} [\cos(\omega_k \Delta_k) + i \sin(\omega_k \Delta_k)].$$

Main Contributions

Its main contributions can be summarized as

Exponential-Trapezoidal Discretization + Complex-Valued SSM + MIMO.

Improvements based on Mamba

$$\mathbf{g}(\mathbf{x}) = \text{TopK} \left(\text{Softmax} \left(\frac{W_g \mathbf{x} + \mathbf{b}_g}{\tau} \right) \right),$$

$$E_i(\mathbf{x}) = \mathbf{W}_{2,i} \cdot \sigma(\mathbf{W}_{1,i} \mathbf{x} + \mathbf{b}_{1,i}) + \mathbf{b}_{2,i},$$

$$\mathbf{y} = \sum_{i=1}^N g_i(\mathbf{x}) \cdot E_i(\mathbf{x})$$

Hybrid Mamba/attention models

